

Audio in Godot 4.6

Schritte, Ambient & UI-Sounds

Projektkontext

Hauptszene: `game.tscn` (Node "Game")

Spieler: `player.tscn` (Node "Girl"), Script `scenes/player.gd`

Welt: `world.tscn`, Script `scripts/events.gd`

Schritt-Sounds: `res://assets/sound/steps/step_1.wav ... step_7.wav`

Animationen: `down, idle, left, right, up`

Kapitel 1 — Audio

Atmosphäre

Ambient-Klängen / orchestrale Soundtrack

Free Music Archive

<https://freemusicarchive.org/search/?quicksearch=epic+choir+>

Soundeffekte

Spielwelten glaubwürdiger. Schritte auf verschiedenen Untergründen, Umgebungsgeräusche, technische Signale oder Naturklänge

Pixabay, z.B. <https://pixabay.com/de/sound-effects/natur-footsteps-on-the-nature-trail-419017/>

Oder Soundeffekte (Plop, Explosion ...):

<https://pixabay.com/sound-effects/search/?order=ec>

OpenGameArt [https://opengameart.org/art-search-](https://opengameart.org/art-search-advanced?keys=walking&title=&field_art_tags_tid_op=or&field_art_tags_tid=&name=&field_art_type_tid%5B%5D=13&sort_by=count&sort_order=DESC&items_per_page=24&Collection=)

[advanced?keys=walking&title=&field_art_tags_tid_op=or&field_art_tags_tid=&name=&field_art_type_tid%5B%5D=13&sort_by=count&sort_order=DESC&items_per_page=24&Collection=](https://opengameart.org/art-search-advanced?keys=walking&title=&field_art_tags_tid_op=or&field_art_tags_tid=&name=&field_art_type_tid%5B%5D=13&sort_by=count&sort_order=DESC&items_per_page=24&Collection=)

FreeSound <https://freesound.org/people/larspriebe/sounds/629003/>

Audiobearbeitung

Kürzen von Audiodateien, Lautstärkeanpassungen, das Entfernen von Störgeräuschen oder das Erstellen von Loops für Hintergrundmusik

Open Source Editors

- Audiomass <https://audiomass.co/>
- Audacity <https://www.audacityteam.org/>

Kapitel 1 — Schrittgeräusche

Ziel: Schritte werden frame-genau an die Laufanimation gekoppelt: Sobald der Fuß aufsetzt (Frame 2 und Frame 5), spielt ein zufälliger Schritt-Sound.

Node-Aufbau

```

Girl (CharacterBody2D)
├─ AnimatedSprite2D
└─ StepSound (AudioStreamPlayer2D)
    └─ Stream: AudioStreamRandomizer
        ├── step_1.wav
        └─ ... step_7.wav
  
```

StepSound ist ein `AudioStreamPlayer2D` — die Position des Sounds richtet sich nach der Spielerposition im 2D-Raum (Stereo-Panning, Lautstärke nach Entfernung).

AudioStreamRandomizer einrichten

Damit nicht bei jedem Schritt derselbe Sound läuft, nutzen wir einen `AudioStreamRandomizer`. Er wählt bei jedem `play()` einen zufälligen Stream aus.

1. StepSound (`AudioStreamPlayer2D`) auswählen.
2. Im Inspector beim Feld **Stream** → **New AudioStreamRandomizer**.
3. Den Randomizer anklicken, um ihn aufzuklappen.
4. Unter **Streams** mit dem **+**-Button alle 7 WAVs einzeln hinzufügen: `step_1.wav` bis `step_7.wav`.
5. Playback Mode auf **Random (Avoid Repeat)** stellen.
6. Optional: **Random Volume Offset dB** leicht erhöhen (z. B. `0.42`) für natürliche Variation.

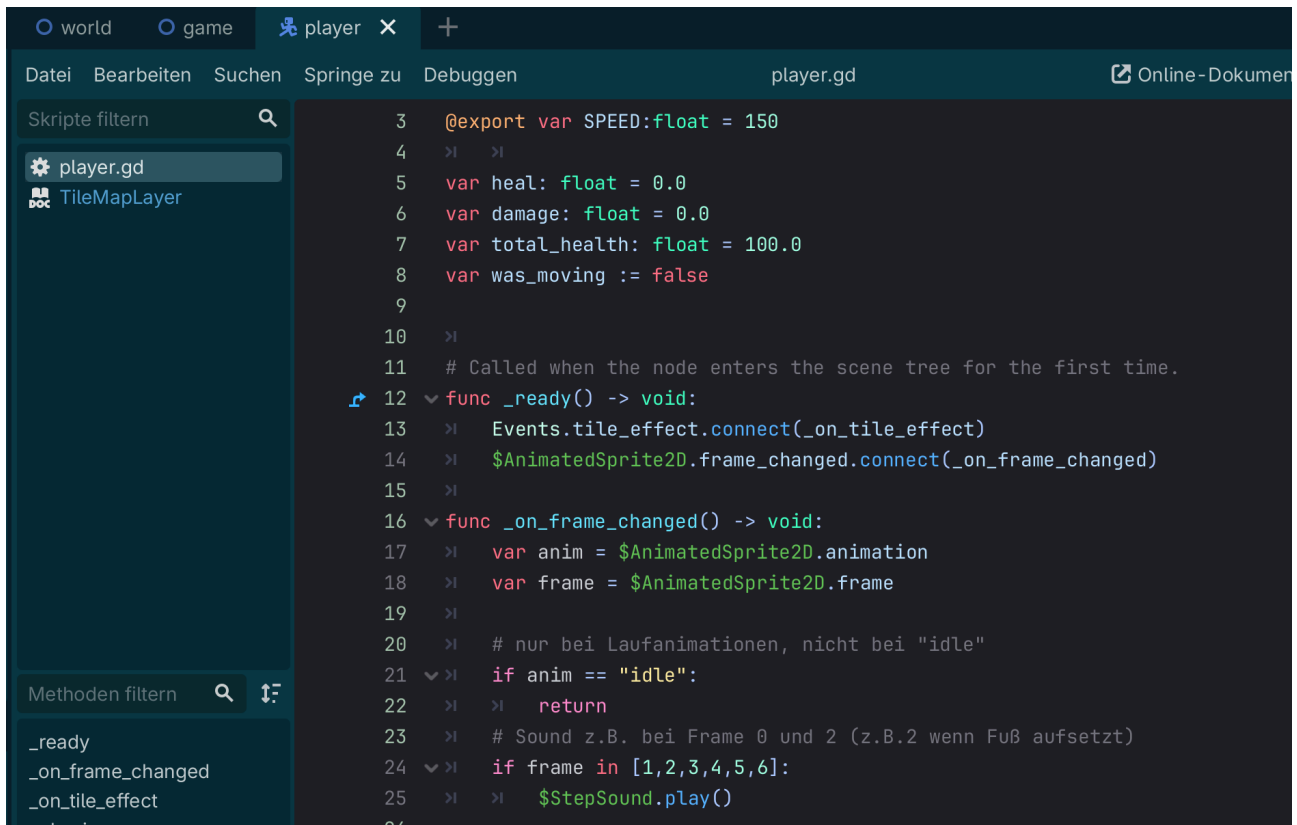


frame_changed-Signal verbinden

Das `frame_changed`-Signal des `AnimatedSprite2D` feuert bei jedem Frame-Wechsel — so bleibt der Sound exakt synchron zur Animation, unabhängig von der FPS-Rate.

Die Verbindung erfolgt im Code (siehe `_ready()`),

```
$AnimatedSprite2D.frame_changed.connect(_on_frame_changed)
```



Handler: Frames

Frame 0 wird explizit ausgeschlossen — sonst würde beim Übergang `Idle`→`Walk` (beide starten auf Frame 0) ein Konflikt entstehen.

was_moving: ersten Schritt manuell auslösen

Problem: `frame_changed` feuert **nicht**, wenn man aus dem Stand losläuft und sowohl "idle" als auch die Laufanimation auf Frame 0 stehen — der Frame ändert sich beim Wechsel nicht. Der allererste Schritt wird also verschluckt.

Lösung: ein `was_moving`-Boolean. Beim Übergang `Stand` → `Laufen` wird der erste Schritt einmalig manuell ausgelöst

```

var is_moving: bool = direction != Vector2.ZERO

# Erster Schritt sofort beim Loslaufen
if is_moving and not was_moving:
    $StepSound.play()
was_moving = is_moving
  
```

Kapitel 2 — Umgebungs-Loops (Ambient)

Ambient-Sounds (Wind, Vögel, Regen) laufen dauerhaft in einer Schleife und schaffen Atmosphäre. Sie sind nicht ortsgebunden, d.h. Verwendung eines nicht-positionalen `AudioStreamPlayer`.

Assets: `res://assets/sound/ambient/wind_loop.ogg`, `birds_loop.ogg`, `rain_loop.ogg`. — `.ogg` eignet sich besser für lange Loops als `.wav` (kleinere Dateien, eingebauter Loop-Punkt).

Node-Aufbau in `world.tscn`

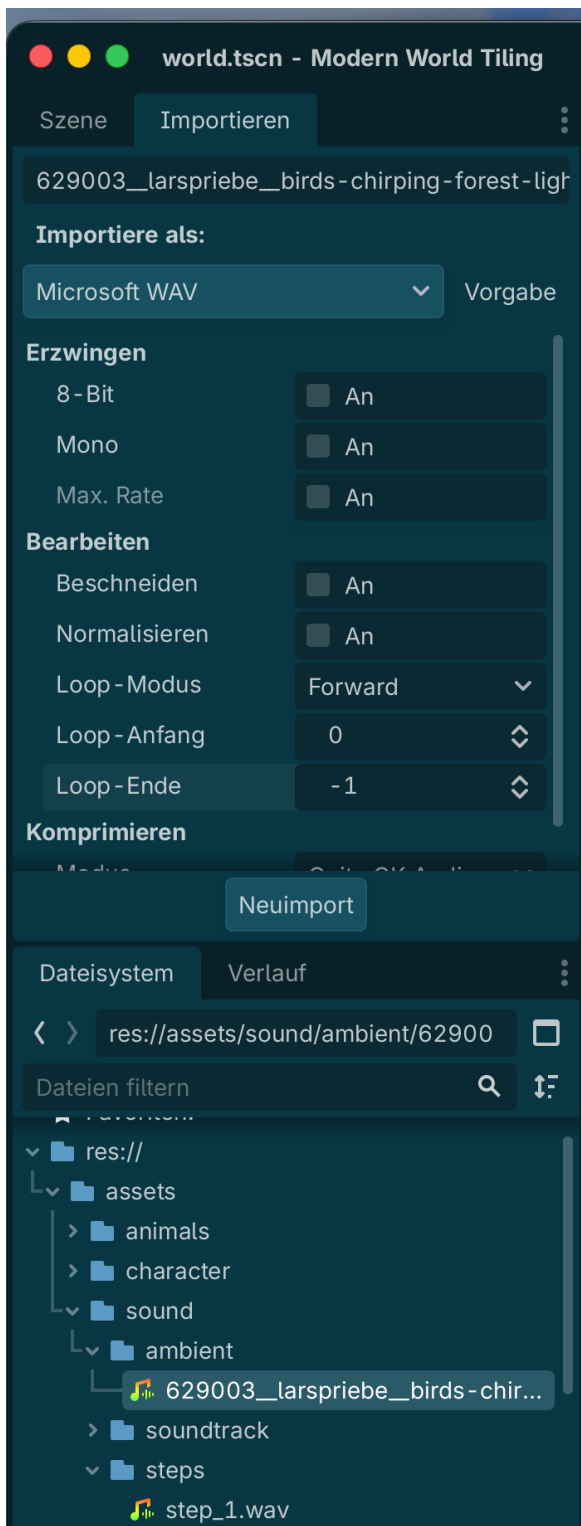
```
World (Node2D)
├─ AmbientWind (AudioStreamPlayer)
├─ AmbientBirds (AudioStreamPlayer)
└─ ZoneDetector (Area2D)
    └─ CollisionShape2D
```

`ZoneDetector` ist eine `Area2D`, die erkennt, wann der Spieler eine Zone betritt — z. B. einen Wald mit Vogelgezwitscher.

autoplay und loop aktivieren

Für Dauerschleifen, die sofort beim Szenenstart laufen sollen (z. B. Wind):

1. `AmbientWind` auswählen, **Stream** auf `wind_loop.ogg` setzen.
2. Im Inspector **Autoplay** = **true** → startet automatisch.
3. Loop aktivieren: Im **Import-Tab** der `.wav`-Datei → **Loop-Modus** auf Forward → **Neuimport**.

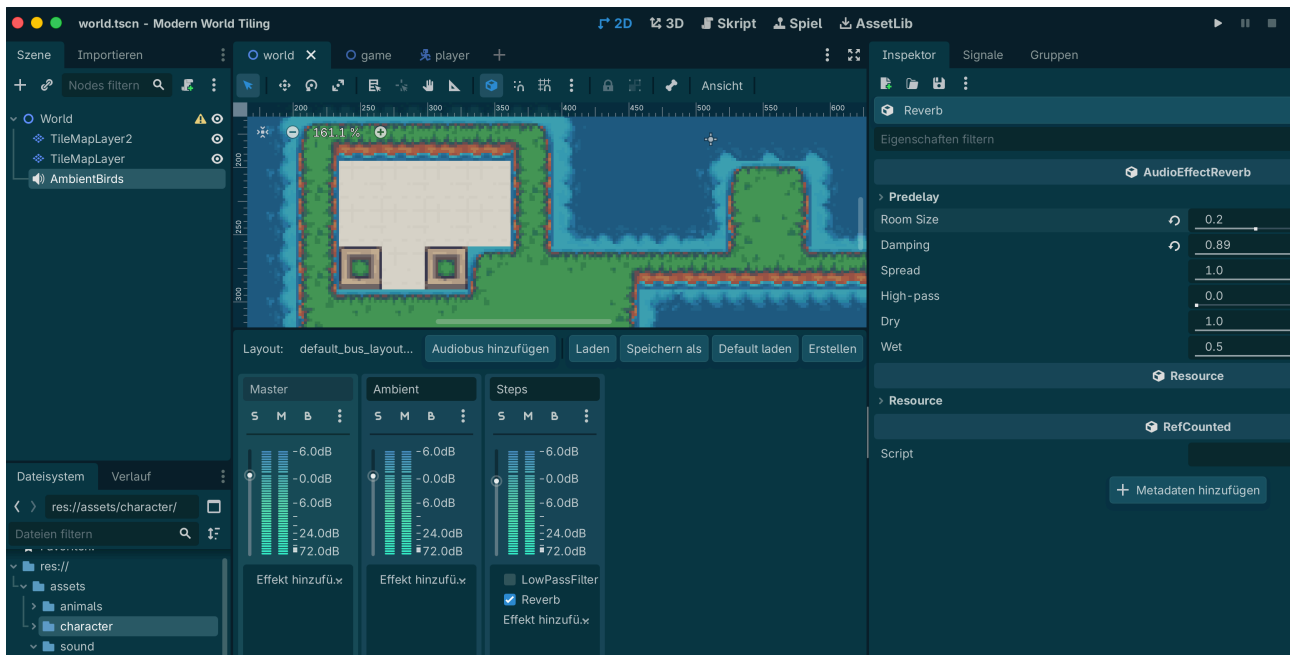


Audio Bus "Ambient" anlegen

Audio-Bus ist ein eingebautes Godot-Konzept auf Signalebene. Ein Bus ist ein Kanal, durch den der Ton fließt, bevor er die Boxen erreicht. Ein Audio Bus kann Sound gruppieren, und gesamtheitlich Effekte, Lautstärke etc einstellen

1. Unten im Editor den **Audio**-Tab öffnen.
2. Add Bus → umbenennen in **Ambient**.
3. Sicherstellen, dass der Ambient-Bus an **Master** ausgibt.

4. Bei `AmbientWind` und `AmbientBirds` im Inspector unter **Bus** den Bus **Ambient** auswählen.



Ambient per Area2D starten/stoppen (scenes/world.gd)

Vogelgezwitscher soll nur im Wald laufen. Die Signale `body_entered` und `body_exited` der `ZoneDetector-Area2D` feuern die entsprechenden Signale, die dann im Autoload Signal Bus für alle Skripte verfügbar sind:

- Vögel sind nur außerhalb des Raumes hörbar
- Schrittgeräusche wechseln außerhalb des Raumes

```

7  ▼ func _ready() -> void:
8      >| >| # Bridge: Area2D signals → bus
9      >| $ZoneDetector.body_entered.connect(_on_room_entered)
10     >| $ZoneDetector.body_exited.connect(_on_room_exited)
11     >| # This script also listens (to play birds)
12     >| Events.zone_entered.connect(_on_zone_entered)
13     >| Events.zone_exited.connect(_on_zone_exited)
14
15     # --- Area2D → bus ---
16  ▼ func _on_room_entered(body: Node2D) -> void:
17     >| Events.zone_entered.emit("forest", body)
18
19  ▼ func _on_room_exited(body: Node2D) -> void:
20     >| Events.zone_exited.emit("forest", body)
21
22     # --- bus → birds ---
23  ▼ func _on_zone_entered(zone_name: String, body: Node2D) -> void:
24     >| >| # entered the room (hall) → birds off>|
25  ▼ >| if body.name == "Girl" and ambient_birds.playing:
26     >| >| ambient_birds.stop()
27
28  ▼ func _on_zone_exited(zone_name: String, body: Node2D) -> void:
29     >| >| # left the room → in the forest → birds on
30  ▼ >| if body.name == "Girl" and not ambient_birds.playing:
31     >| >| ambient_birds.play()

```

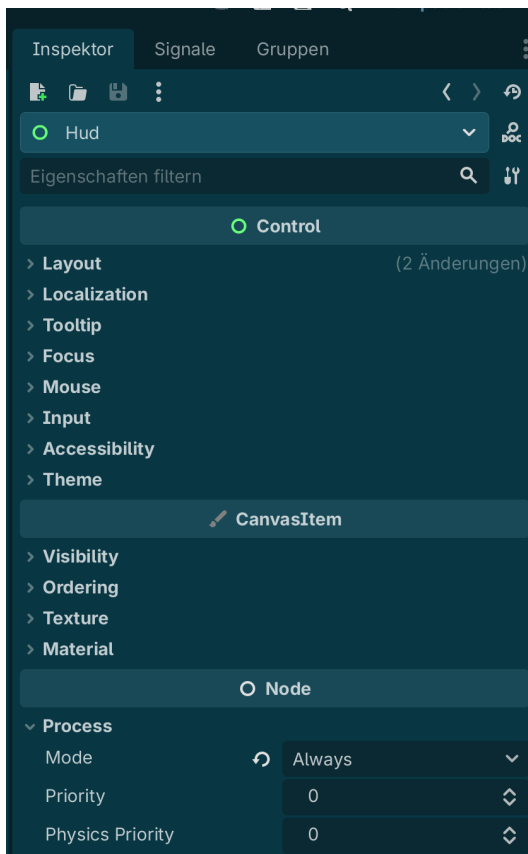
Kapitel 3 — UI-Sounds

UI-Sounds (Klick, Hover, Bestätigung) sind nicht im Spielraum verortet

HUD anlegen

Wichtig sind drei Sachen:

- Control Node (grün)
- Canvas Layer muss UI Elemente beherbergen
- Im Inspektor muss bei ausgewähltem ‚HUD‘ Node, Process auf ‚Always‘ gesetzt werden



Hier noch die entsprechende Logik, um den UI Sound per Signal auch mit dem UI Sound zu verknüpfen ...

